

UDS 软件实现定义的应用层接口

除了诊断服务之外，ISO-14229-1 中还描述了调用这些诊断服务的函数接口，即将具体诊断服务的实现封装在一个个的 API 中，做协议实现的公司负责实现具体的 API，使用这些协议的公司只需要调用这些 API 就可以了。

为了使 API 的调用更简单明了，UDS 中规定了 6 种通信原语（primitive），每个诊断服务的实现都包括了这 6 种原语，而且原语的格式都是一致的，所以对调用者来说比较简单。



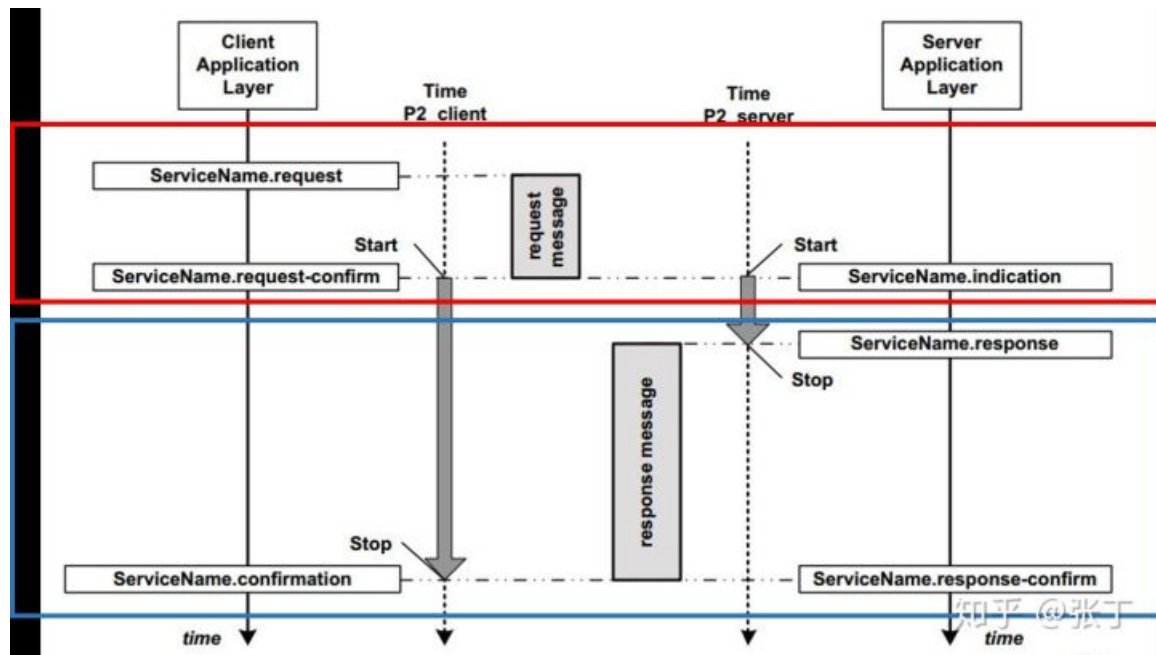
原语的实现与调用

如上图所示，原语，也就是 API 的实现在 UDS 协议栈的应用层中，用户自己编写的软件调用这些 API 来实现通信，上图的左右两部分分别表示参与通信的两端。

下面是具体的 6 个原语：

1. service request primitive
2. service request-confirmation primitive
3. service indication primitive
4. service response primitive
5. service response-confirmation primitive
6. service confirmation primitive

前 3 个用于发送请求（下图红框），后 3 个用于发送响应（下图蓝框）。



UDS 应用层服务原语

service request primitive: client 的用户软件调用这个原语进行诊断请求的发送。

service request-confirm primitive: client 的应用层软件使用这个原语通知 client 的用户软件诊断请求已经发送到总线上。

service indication primitive: server 的应用层软件使用这个原语通知 server 的用户软件有诊断请求到达。

service response primitive: server 的用户软件调用这个原语进行诊断响应的发送。

service response-confirm primitive: server 的应用层软件使用这个原语通知 server 的用户软件诊断响应已经发送到总线上。

service confirmation primitive : client 的应用层软件使用这个原语通知 client 的用户软件有诊断响应到达。

从上图可以看出，这 6 个原语中，1-4，2-5，3-6 其实作用一样，只是使用方向相反，一个用于请求，一个用于响应。

注：结合上图，就可以更清晰地理解我在上一篇文章中提到的 P2client 和 P2_server 这两个时间参数的起止点了。

以上这 6 种原语，描述格式都是这样的：

`service_name.type ( parameter A, parameter B, parameter C [,parameter 1, ...] )`

service_name 指代具体的诊断服务，比如 SessionControl , ECURest 等等，而 type 就是用于描述上边的 6 种原语之一。比如，要定义一个发送 SessionControl 请求的 API，那么它的格式就是 SessionControl.request(parameter A, parameter B, parameter C [,parameter 1, ...])，这样就很容易看出来各个 API 的含义了。

下面把所有原语包含参数的格式列举一下。

service request primitive:

```
service_name.request (  
A_MType,  
A_SA,  
A_TA,  
A_TA_type,  
[A_AE],  
A_Length,  
A_Data[,parameter 1, ...],  
)
```

service request-confirmation primitive:

```
service_name.req_confirm (  
A_Mtype,  
A_SA,  
A_TA,  
A_TA_type,  
[A_AE],  
A_Result  
)
```

service indication primitive:

```
service_name.indication (  
A_MType,  
A_SA,  
A_TA,  
A_TA_type,  
[A_AE],
```

```
A_Length  
A_Data[,parameter 1, ...],  
)
```

service response primitive :

```
service_name.response (  
A_Mtype,  
A_SA,  
A_TA,  
A_TA_type,  
[A_AE],  
A_Length  
A_Data[,parameter 1, ...],  
)
```

service response-confirmation primitive :

```
service_name.rsp_confirm (  
A_Mtype,  
A_SA,  
A_TA,  
A_TA_type,  
[A_AE],  
A_Result  
)
```

service confirmation primitive :

```
service_name.confirm (  
A_Mtype,  
A_SA,  
A_TA,  
A_TA_type,  
[A_AE],  
A_Length  
A_Data[,parameter 1, ...],
```

)

诊断服务名.原语类型 这种格式就构成了调用所有诊断服务的通用格式。

所有的原语都有以下几个参数：

A_Mtype：枚举类型，有两种取值，分别是 **diagnostics** 和 **remote diagnostics**，**remote diagnostics** 我没接触过，所以在这里不误导大家了。**diagnostics** 就表示这是一个诊断报文。

A_SA：源地址，2 字节无符号整型，取值范围 **0x0000 – 0xFFFF**

A_TA：目的地址，2 字节无符号整型，取值范围 **0x0000 – 0xFFFF**

注：源地址和目的地址是在车型的网络拓扑或者通信矩阵中为各个 **ECU** 规定的逻辑地址，这个地址可以用于诊断目的，也可以用于其他目的（比如网络管理）。

A_TA_type：对 **A_TA** 的扩展，其实就是额外描述一下 **A_TA**，是功能寻址还是物理寻址。

[A_AE]：只有当 **A_Mtype == remote diagnostics** 的时候才使用。

另外几个 **result** , **length** , **data** 之类的参数，看名字就知道什么意思了，不再赘述。